

NFT — Non-Fungible Tokens

Concepts, Architecture, Standards, Use Cases, Solidity and Hardhat 3 Practical Guide

1. Introduction to NFT

NFT stands for Non-Fungible Token. An NFT is a blockchain-based token that represents a unique digital item, record, right, or identifier. Unlike a cryptocurrency unit such as 1 ETH, where one unit is normally interchangeable with another unit of the same currency, each NFT can have its own token ID, metadata, ownership history and characteristics.

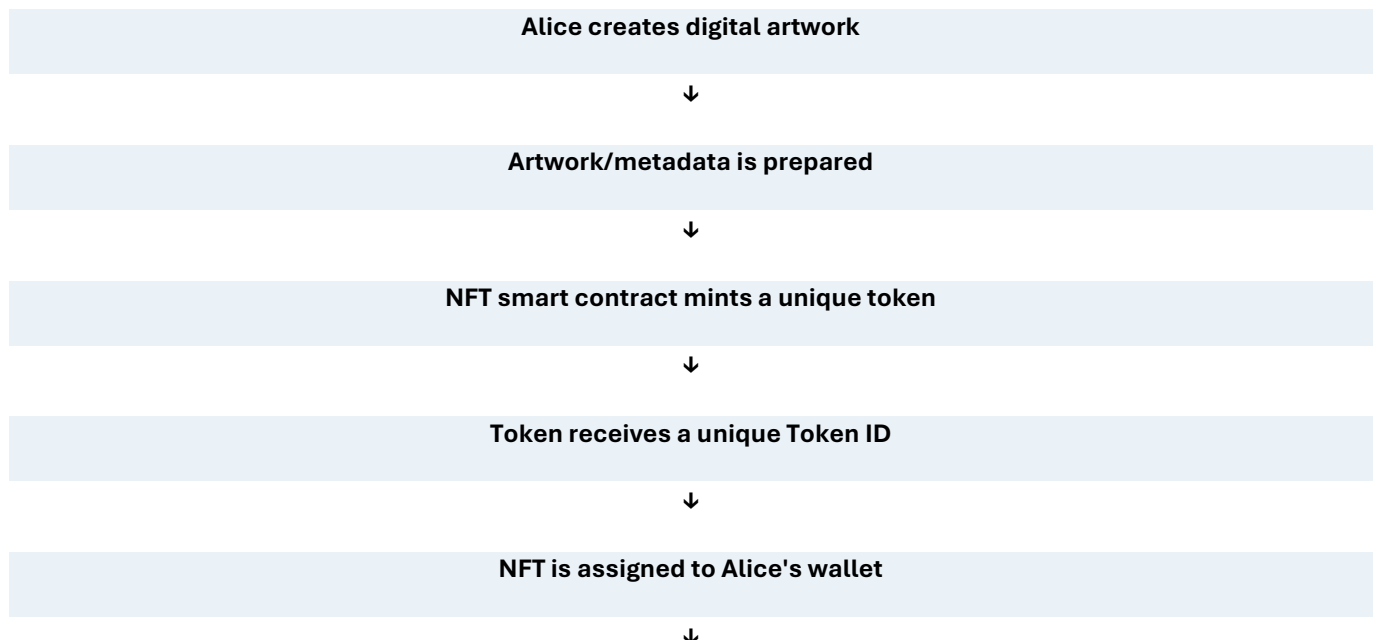
NFTs are commonly associated with digital art and collectibles, but the underlying technology can also be used for gaming assets, certificates, tickets, memberships, licenses, domain names, event passes, and other applications where uniqueness and verifiable ownership are useful.

2. What Does 'Fungible' Mean?

Property	Fungible Asset	Non-Fungible Asset / NFT
Meaning	One unit can normally be exchanged for another equivalent unit.	Each item is distinguishable or unique.
Examples	₹100 note, ETH, BTC, USDC	Digital artwork NFT, certificate NFT, game item, ticket
Interchangeability	High	Usually not interchangeable on a one-to-one basis
Identity	Units of the same type are equivalent for accounting purposes.	Each token can have a unique token ID and metadata.
Typical token standard	ERC-20	ERC-721 or ERC-1155

3. Simple Example

Consider a digital artist named Alice who creates an original digital painting. She can mint an NFT representing that work.



NFT can later be transferred or sold

The blockchain can record the token's ownership and transfer history. The NFT itself usually contains or points to metadata describing the associated item.

4. NFT Architecture



The wallet signs transactions. The DApp provides the interface. The smart contract implements minting, ownership and transfer rules. The blockchain stores the token state and transaction history. Metadata commonly contains information such as the NFT name, description, image reference and attributes.

5. Major Components of an NFT

Component	Description	Example
Smart Contract	Defines NFT behavior such as minting, ownership and transfers.	ERC-721 contract
Token ID	Unique identifier for a token within a contract.	Token #1, Token #250
Owner Address	Blockchain address currently owning the token.	0xAB...1234
Metadata	Descriptive information about the NFT.	Name, description, attributes
Token URI	URI used to locate metadata.	ipfs://... or HTTPS URI
Media	Associated image, video, audio or other asset.	PNG, MP4, MP3, 3D asset
Blockchain	Network where the NFT contract exists.	Ethereum or another EVM network

6. How NFT Minting Works

Minting is the process of creating a new NFT token through a smart contract.



Minting does not automatically mean that the entire image or video is stored directly on-chain. In many implementations, the smart contract stores a token URI that points to metadata, and the metadata in turn references the media.

7. NFT Metadata

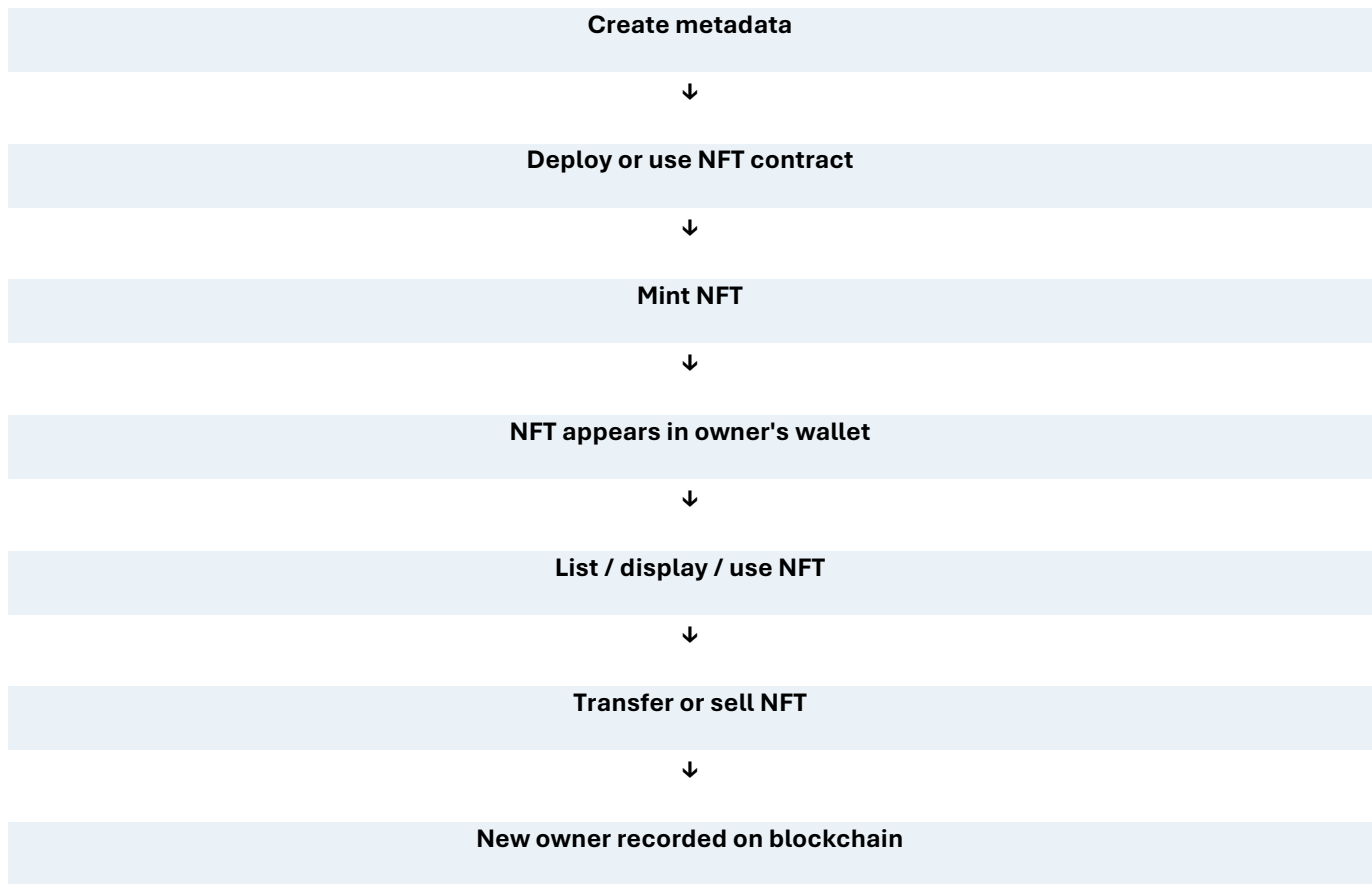
A typical NFT metadata document may be represented in JSON:

```
{
  "name": "My First NFT",
  "description": "A sample NFT created for blockchain training",
  "image": "ipfs://<content-identifier>/art.png",
  "attributes": [
    {
      "trait_type": "Course",
      "value": "Blockchain"
    }
  ]
}
```

The exact metadata conventions depend on the application and ecosystem. IPFS is frequently used as a decentralized content-addressed storage option, although HTTPS and other storage approaches are also possible.

8. NFT Lifecycle





9. ERC-721 Standard

ERC-721 is a widely used Ethereum token standard for non-fungible tokens. Each token has a unique token ID and an owner. The standard defines common functions and events so wallets, marketplaces and applications can interact with NFTs consistently.

Function / Concept	Purpose
balanceOf(owner)	Returns the number of NFTs owned by an address.
ownerOf(tokenId)	Returns the owner of a particular token.
safeTransferFrom(...)	Safely transfers an NFT between addresses.
transferFrom(...)	Transfers an NFT subject to authorization rules.
approve(...)	Approves another address to transfer a particular NFT.
setApprovalForAll(...)	Authorizes an operator for all NFTs of an owner.
Transfer event	Emitted when ownership changes.
Approval event	Emitted when token approval changes.

10. ERC-1155 Standard

ERC-1155 is a multi-token standard. A single smart contract can manage multiple token types, including fungible, non-fungible and semi-fungible-style assets. It is especially useful in games and applications containing many different asset types.

Feature	ERC-721	ERC-1155
Primary model	One unique NFT per token ID	Multiple token types in one contract
Quantity per ID	Typically one unique token	Can support multiple units of an ID
Batch transfers	Not the core model	Supported by the standard
Typical use	Art, certificates, unique collectibles	Games, inventory systems, mixed assets
Contract efficiency	Simple unique-token model	Can be efficient for many asset classes

11. NFT Ownership and Transfer

An NFT smart contract maintains ownership information. A transfer changes the owner recorded by the contract.



Ownership of an NFT token does not automatically grant copyright, trademark rights, commercial rights, or ownership of an external physical object. Those rights depend on the applicable license, contract and law.

12. NFT Marketplaces

NFT marketplaces provide interfaces for discovering, listing, buying and selling NFTs. Examples include OpenSea and other marketplace platforms. A marketplace typically interacts with users' wallets and blockchain smart contracts rather than replacing the underlying blockchain ownership record.



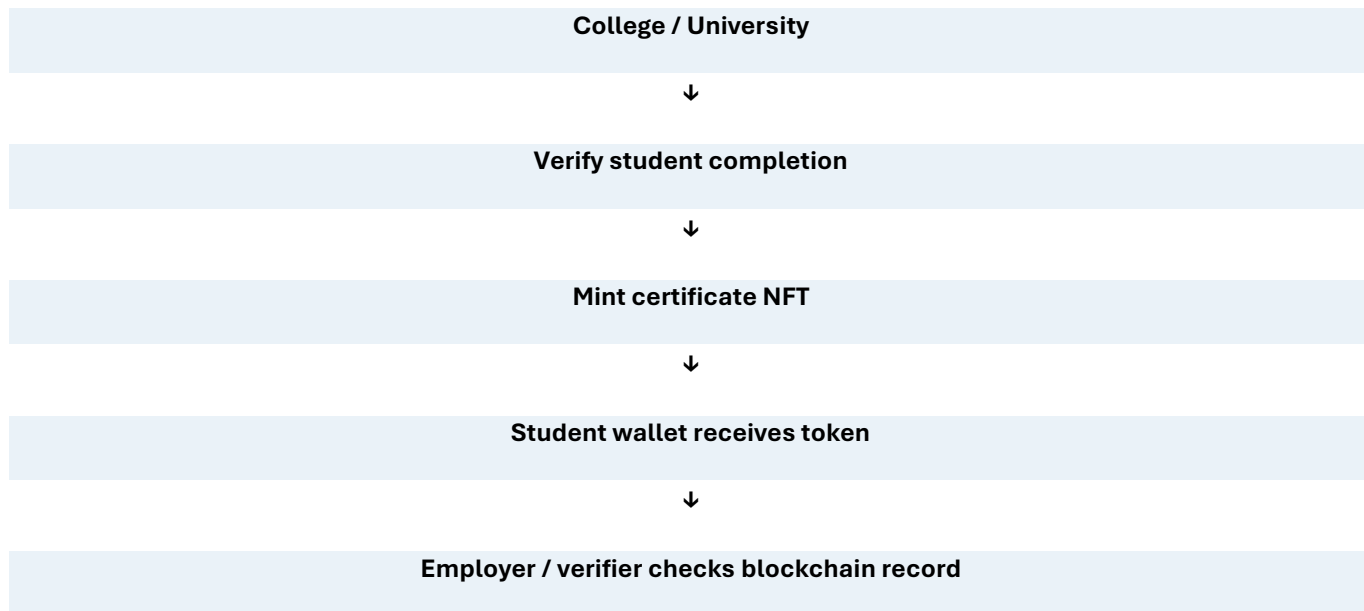
13. Real-World NFT Use Cases

Area	Possible NFT Use	Example
Digital Art	Represent unique digital works and provenance.	Artist mints an artwork NFT.

Gaming	Represent in-game items.	Sword, skin, character or land.
Education	Represent verifiable credentials.	Course completion certificate.
Events	Represent unique tickets or passes.	Conference ticket NFT.
Membership	Represent access rights.	Club/community membership.
Collectibles	Represent limited digital collectibles.	Sports or character collectible.
Music / Media	Represent editions or access rights.	Limited digital release.
Domains / Identity	Represent blockchain-native names or identifiers.	Blockchain domain record.

14. NFT Use Case in Education

NFT-style tokens can be explored for academic certificates where uniqueness and verifiable issuance are useful. A college can issue a token representing a certificate to a student's wallet.



For real deployments, privacy, revocation/correction, legal recognition, wallet recovery, data protection, accessibility and institutional governance must be designed carefully. Sensitive student information should not simply be placed on a public blockchain.

15. Advantages and Limitations

Potential Advantage	Important Limitation / Risk
Verifiable token ownership	Ownership of token may not equal copyright or legal ownership of associated content.
Programmable transfers and permissions	Smart-contract bugs can cause losses or unexpected behavior.
Transparent transaction history	Public blockchains can create privacy concerns.
Interoperability through standards	Wallet/marketplace support can vary.
Digital scarcity / uniqueness	Scarcity does not guarantee economic value.
Global blockchain access	Network fees and user experience can be barriers.
Composable applications	External metadata/storage can become unavailable if poorly designed.

16. NFT Security Risks

- Phishing and malicious wallet-signature requests.
- Loss or theft of wallet private keys.
- Smart-contract vulnerabilities.
- Fake or copied NFT collections.
- Metadata or media hosted at unreliable locations.

- Marketplace scams and malicious links.
- Incorrect approval permissions that allow unwanted transfers.
- Price volatility and speculative financial risk.

17. Simple ERC-721 Solidity Example

The following teaching example uses OpenZeppelin's ERC-721 implementation. It is intentionally simple and should not be treated as a complete production NFT system.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract StudentNFT is ERC721, Ownable {
    uint256 private _nextTokenId;

    constructor()
        ERC721("StudentNFT", "SNFT")
        Ownable(msg.sender)
    {}

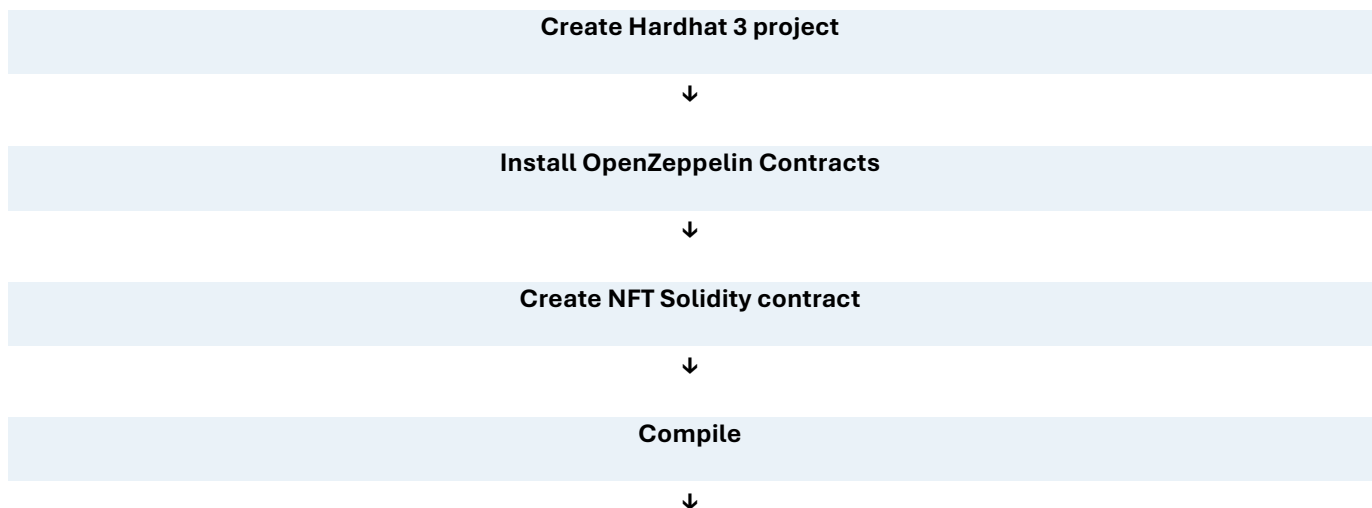
    function mint(address to) public onlyOwner returns (uint256) {
        uint256 tokenId = _nextTokenId;
        _nextTokenId++;

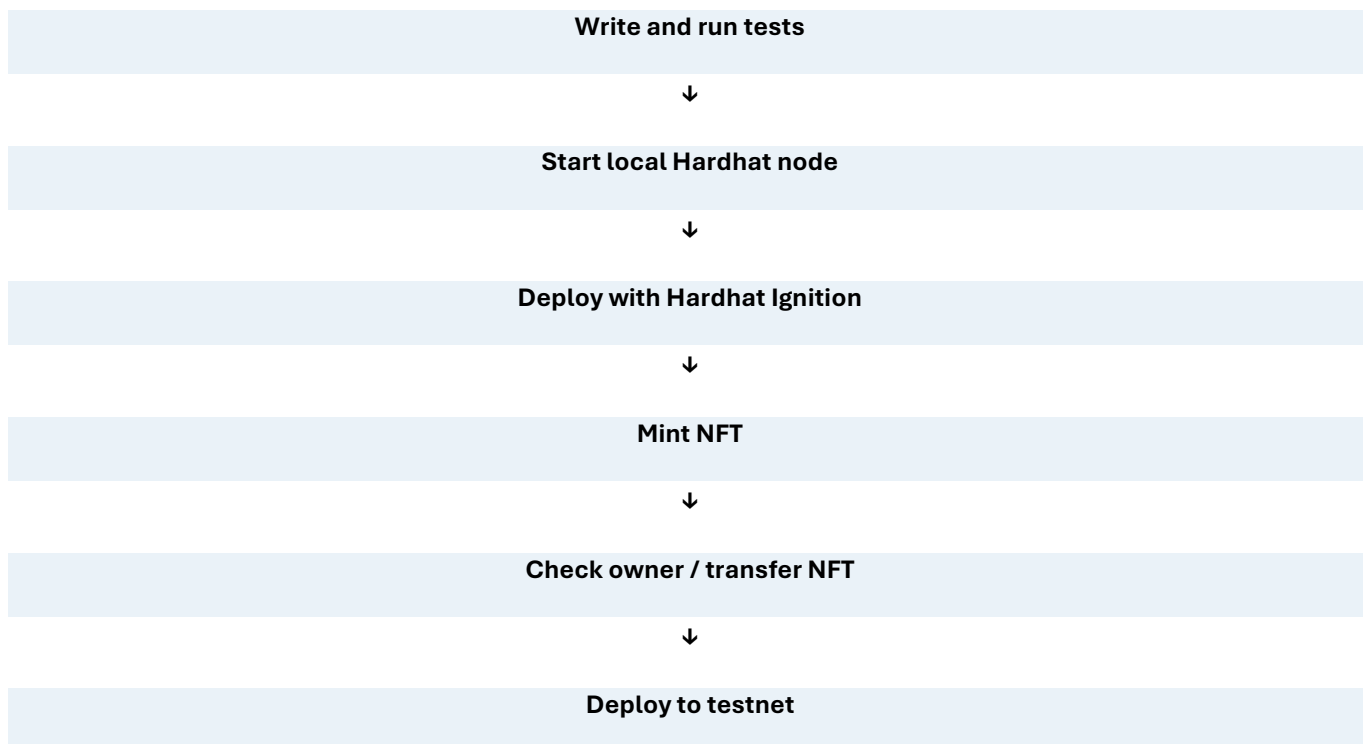
        _safeMint(to, tokenId);
        return tokenId;
    }
}
```

In this example, ERC721 provides the standard NFT behavior. Ownable restricts minting to the contract owner. `_safeMint` creates the token and assigns it to the specified wallet.

18. NFT Development with Hardhat 3

A typical learning workflow using Hardhat 3 is:





19. Important Hardhat 3 Commands for the NFT Project

Command	Purpose
<code>npx hardhat --init</code>	Initialize a Hardhat project.
<code>npm install @openzeppelin/contracts</code>	Install reusable OpenZeppelin smart-contract implementations.
<code>npx hardhat compile</code>	Compile the NFT contract.
<code>npx hardhat test</code>	Run the configured tests.
<code>npx hardhat node</code>	Start the local Hardhat blockchain node.
<code>npx hardhat ignition deploy ignition/modules/StudentNFT.ts --network localhost</code>	Deploy the NFT using an Ignition module to localhost.
<code>npx hardhat console --network localhost</code>	Open a console connected to the local node.
<code>npx hardhat ignition deploy ignition/modules/StudentNFT.ts --network sepolia</code>	Deploy the NFT module to Sepolia, when network configuration is ready.
<code>npx hardhat --help</code>	Show tasks available in the current Hardhat project.

20. Suggested Hardhat 3 NFT Project Structure

```
nft-project/
├── contracts/
│   └── StudentNFT.sol
├── test/
│   └── StudentNFT.ts
├── ignition/
│   └── modules/
│       └── StudentNFT.ts
├── metadata/
│   └── student-certificate.json
├── hardhat.config.ts
└── package.json
```

└─ artifacts/
└─ cache/

21. Suggested Practical Lab: Student Certificate NFT

Objective: Build a simple NFT-based certificate prototype for teaching purposes.

1. Initialize a Hardhat 3 project.
2. Install OpenZeppelin Contracts.
3. Create StudentNFT.sol using ERC-721.
4. Compile the smart contract.
5. Create tests for minting, ownership and transfers.
6. Start a local Hardhat node.
7. Create an Ignition deployment module.
8. Deploy StudentNFT to localhost.
9. Mint a certificate NFT to a student wallet.
10. Use ownerOf(tokenId) to verify ownership.
11. Transfer the NFT in a controlled demonstration.
12. Extend the project with metadata and tokenURI support.
13. Optionally deploy the learning project to Sepolia.

22. NFT vs Cryptocurrency

NFT	Cryptocurrency / Fungible Token
Usually unique or distinguishable by token ID.	Units of the same asset are generally interchangeable.
Common standards: ERC-721, ERC-1155.	Common fungible standard: ERC-20.
Often represents a unique item, record or right.	Often used as currency, utility, governance or value-transfer token.
Example: certificate token #25.	Example: 1 USDC or a unit of an ERC-20 token.

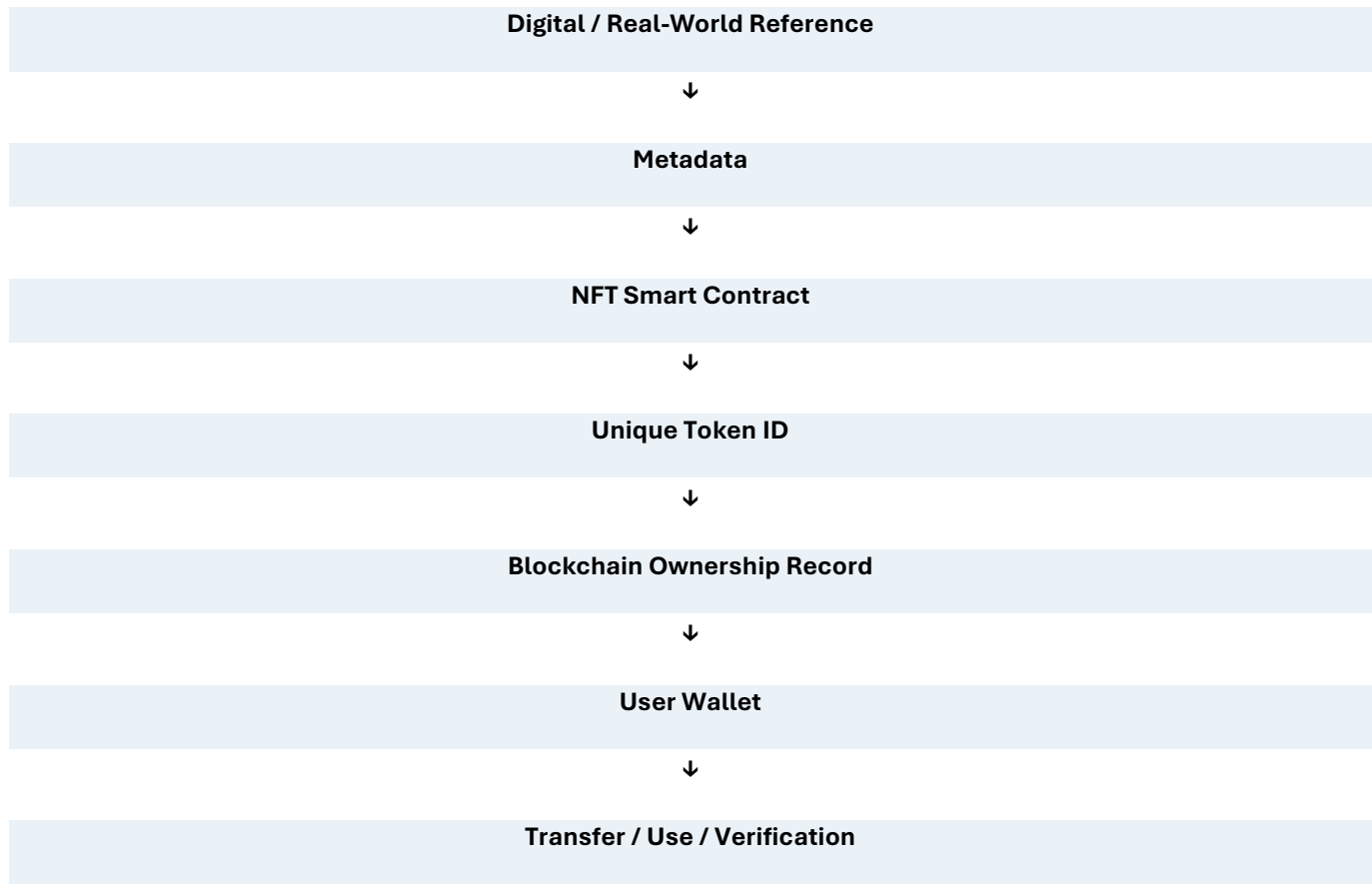
23. Key Terms for Students

Term	Meaning
NFT	Non-Fungible Token.
Minting	Creating a new token through a smart contract.
Token ID	Identifier of a token within an NFT contract.
Metadata	Descriptive information associated with the NFT.
Token URI	URI used to locate token metadata.
Wallet	Software/hardware used to manage blockchain accounts and sign transactions.
Marketplace	Application used to discover/list/trade NFTs.
ERC-721	Ethereum standard for unique NFTs.
ERC-1155	Ethereum multi-token standard.
IPFS	Content-addressed distributed storage/network commonly used for NFT media/metadata.
Gas	Blockchain transaction execution fee on networks such as Ethereum.

24. Simple Definition for Examination / Classroom Use

NFT (Non-Fungible Token) is a blockchain-based token used to represent a unique digital item, record, right or identifier. Each NFT can have a distinct token ID, metadata and ownership record, and smart contracts control how it is created and transferred.

25. Summary Diagram



26. Official Learning References

Hardhat documentation: <https://hardhat.org/docs>

OpenZeppelin Contracts documentation: <https://docs.openzeppelin.com/contracts/>

Ethereum NFT documentation: <https://ethereum.org/developers/docs/standards/tokens/erc-721/>

IPFS documentation: <https://docs.ipfs.tech/>

Note: Tooling, network configuration, APIs and marketplace behavior change over time. For practical labs, use the current documentation for Hardhat 3 and the libraries installed in your project.